Food Delivery System

Phase 1

BACSE202 - Database Systems

Aaron Robertson | 25BCE2640

Ark Khandelwal | 25BCE2094

Priyansh Asati | 25BDS0268

Ansh Kapoor | 25BCE2281

EER Diagram Link

EER ALTERNATE LINK

Problem statement and requirements

Food delivery services need consistent records connecting customers, restaurants and delivery partners. Repeating profile and menu details across orders causes conflicting updates. This database manages menus, orders, payments and delivery assignments while preserving each order's agreed prices and captured destination.

Scope and business rules

One customer and one restaurant per order.

One or more items, with repeated portions combined in quantity.

At most one payment and one delivery.

Users can hold customer and partner roles.

Positive quantities/prices, nonnegative fees.

Outside scope: coupons, refunds, GPS tracking and payment-gateway integration.

Users and functional requirements

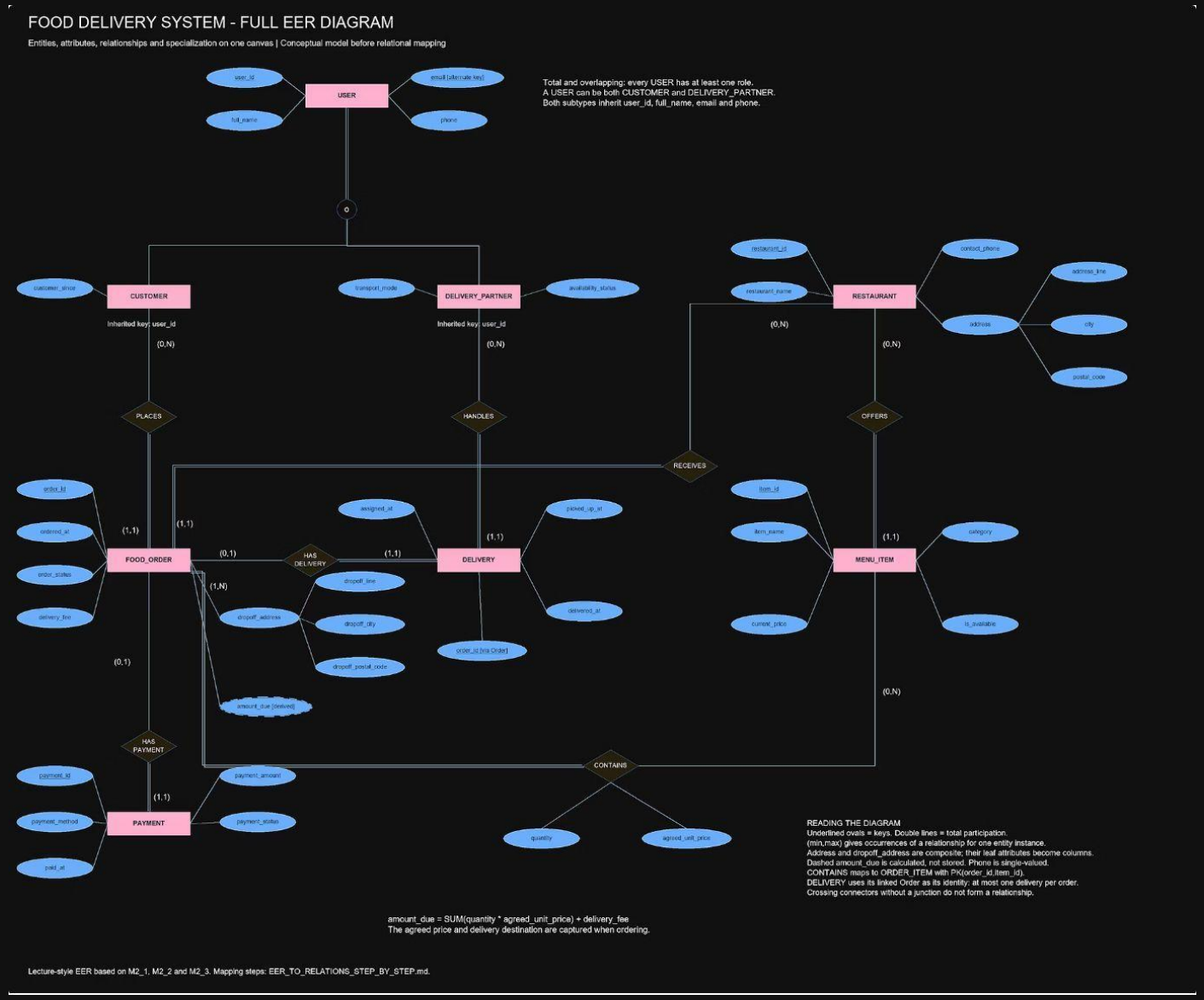
Customer: browse menus, place orders, view totals and order history.

Restaurant operator: maintain menu items and availability, accept and prepare orders.

Delivery partner: view assignments, record pickup and delivery.

Administrator: maintain restaurant records and assign delivery partners.

Complete EER diagram and cardinalities



Relationship participation

PLACES
Customer (0,N), FoodOrder (1,1)

RECEIVES
Restaurant (0,N), FoodOrder (1,1)

OFFERS
Restaurant (0,N), MenuItem (1,1)

HANDLES
Partner (0,N), Delivery (1,1)

CONTAINS (M:N)
FoodOrder (1,N), MenuItem (0,N)

HAS PAYMENT
FoodOrder (0,1), Payment (1,1)

HAS DELIVERY
FoodOrder (0,1), Delivery (1,1)

Total: every USER has a role.

Overlapping: both roles are allowed.

EER Diagram Link

EER ALTERNATE LINK

Relational mapping: accounts and catalog

PK = primary key FK = foreign key UQ = required unique key

Mapped relation	All attributes and keys
APP_USER	user_id PK, full_name, email UQ, phone
CUSTOMER	user_id PK/FK → APP_USER.user_id, customer_since
DELIVERY_PARTNER	user_id PK/FK → APP_USER.user_id, transport_mode, availability_status
RESTAURANT	restaurant_id PK, restaurant_name, address_line, city, postal_code, contact_phone
MENU_ITEM	item_id PK, restaurant_id FK → RESTAURANT.restaurant_id, item_name, category, current_price, is_available

Entity keys become table PKs. Composite addresses become component columns.

Total/overlapping roles use APP_USER plus shared-key subtype tables.

OFFERS (1:N) places restaurant_id in MENU_ITEM.

Relational mapping: orders and transactions

Mapped relation	All attributes and keys
FOOD_ORDER	order_id PK, customer_id FK → CUSTOMER.user_id, restaurant_id FK → RESTAURANT.restaurant_id, ordered_at, order_status, dropoff_line, dropoff_city, dropoff_postal_code, delivery_fee
ORDER_ITEM	order_id PK-part/FK → FOOD_ORDER.order_id, item_id PK-part/FK → MENU_ITEM.item_id, quantity, agreed_unit_price Composite PK (order_id, item_id)
PAYMENT	payment_id PK, order_id UQ/FK → FOOD_ORDER.order_id, payment_method, payment_amount, payment_status, paid_at
DELIVERY	order_id PK/FK → FOOD_ORDER.order_id, partner_id FK → DELIVERY_PARTNER.user_id, assigned_at, picked_up_at, delivered_at

1:N adds FKs. CONTAINS creates ORDER_ITEM. Optional 1:1 uses UQ or shared PK/FK.

Missing rows represent optional payment/delivery. The derived total is omitted.

EER attributes, keys and specialization

USER (user_id, full_name, email, phone)

CUSTOMER (customer_since)

DELIVERY_PARTNER (transport_mode, availability_status)

Both subtypes inherit user_id from USER. USER.email is a required alternate key.

RESTAURANT (restaurant_id, restaurant_name, address(address_line, city, postal_code), contact_phone)

MENU_ITEM (item_id, item_name, category, current_price, is_available)

FOOD_ORDER (order_id, ordered_at, order_status, delivery_fee, dropoff_address(dropoff_line, dropoff_city, dropoff_postal_code), amount_due [derived])

PAYMENT (payment_id, payment_method, payment_amount, payment_status, paid_at)

DELIVERY (order_id [via FoodOrder], assigned_at, picked_up_at, delivered_at)

CONTAINS (quantity, agreed_unit_price)

amount_due = SUM(quantity × agreed_unit_price) + delivery_fee. It is calculated, not stored.

Normalization

UNF to 1NF: repeating groups

Before: repeating item group

ORDER_FORM (order_id, ordered_at, order_status,
customer_id, customer_name, customer_email,
customer_phone, customer_since, restaurant_id,
restaurant_name, restaurant_address_line,
restaurant_city, restaurant_postal_code,
restaurant_phone, dropoff_line, dropoff_city,
dropoff_postal_code, delivery_fee,
(item_id, item_name, category,
current_price, is_available, quantity,
agreed_unit_price)*)

The nested (attributes)* group repeats
for each item in the order.

After: atomic 1NF relations

ORDER_1 (order_id, ordered_at, order_status,
customer_id, customer_name, customer_email,
customer_phone, customer_since, restaurant_id,
restaurant_name, restaurant_address_line,
restaurant_city, restaurant_postal_code,
restaurant_phone, dropoff_line, dropoff_city,
dropoff_postal_code, delivery_fee)

ORDER_LINE_1 (order_id, item_id, item_name,
category, current_price, is_available,
quantity, agreed_unit_price)

Move the item group and copy order_id.

Name the new relation and select its key.

Each cell now contains one value.

1NF to 2NF: partial dependencies

Before: a partial dependency

ORDER_LINE_1 (order_id, item_id, item_name, category, current_price, is_available, quantity, agreed_unit_price)

item_id → item_name, category, current_price, is_available

(order_id, item_id) → quantity, agreed_unit_price

Menu facts need only item_id. Quantity and the agreed price need the complete composite key.

After: 2NF relations

MENU_ITEM_2 (item_id, item_name, category, current_price, is_available)

ORDER_ITEM_2 (order_id, item_id, quantity, agreed_unit_price)

ORDER_2 (order_id, ordered_at, order_status, customer_id, customer_name, customer_email, customer_phone, customer_since, restaurant_id, restaurant_name, restaurant_address_line, restaurant_city, restaurant_postal_code, restaurant_phone, dropoff_line, dropoff_city, dropoff_postal_code, delivery_fee)

Copy item_id as the menu key. Rename the line. ORDER_2 keeps all 18 header facts.

2NF to 3NF: transitive dependencies

Before: ORDER_2

ORDER_2 (order_id, ordered_at, order_status,
customer_id, customer_name, customer_email,
customer_phone, customer_since, restaurant_id,
restaurant_name, restaurant_address_line,
restaurant_city, restaurant_postal_code,
restaurant_phone, dropoff_line, dropoff_city,
dropoff_postal_code, delivery_fee)

order_id → customer_id → customer details

order_id → restaurant_id → restaurant details

Move the descriptions to their own relations. Copy their determining IDs as keys, and retain those IDs in the order.

After: 3NF ordering relations

CUSTOMER_PROFILE_3 (customer_id,
customer_name, customer_email,
customer_phone, customer_since)

RESTAURANT_3 (restaurant_id, restaurant_name,
address_line, city, postal_code, contact_phone)

FOOD_ORDER_3 (order_id, customer_id,
restaurant_id, ordered_at, order_status,
dropoff_line, dropoff_city,
dropoff_postal_code, delivery_fee)

MENU_ITEM_2 and ORDER_ITEM_2 remain unchanged. Destination and fee remain order-specific.

Final normalized schema

APP_USER (user_id, full_name, email, phone)

CUSTOMER (user_id, customer_since)

DELIVERY_PARTNER (user_id, transport_mode, availability_status)

RESTAURANT (restaurant_id, restaurant_name, address_line, city, postal_code, contact_phone)

MENU_ITEM (item_id, restaurant_id, item_name, category, current_price, is_available)

FOOD_ORDER (order_id, customer_id, restaurant_id, ordered_at, order_status, dropoff_line, dropoff_city, dropoff_postal_code, delivery_fee)

ORDER_ITEM (order_id, item_id, quantity, agreed_unit_price)

PAYMENT (payment_id, order_id, payment_method, payment_amount, payment_status, paid_at)

DELIVERY (order_id, partner_id, assigned_at, picked_up_at, delivered_at)

1NF: atomic values. 2NF: whole-key dependence. 3NF: no violating transitive FDs.